

# Service mesh на примере Istio

## Microservice Architecture

REC Проверить, идет ли запись

# Меня хорошо видно & слышно?





# Вадим Перепелкин

- Более 15 лет в ИТ
- Занимал разные позиции (developer, team lead, architect)

@VPerepelkin

vup@rambler.ru

# Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе
#**канал группы**



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

## Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом



Документ



Ответьте себе или
задайте вопрос

# Карта курса


Базовые паттерны микросервисных архитектур
Инфраструктура микросервисов
Инструменты наблюдаемости
Коммуникационные паттерны
Распределенные системы и хранилища
Проектная работа

# Маршрут вебинара

- Service mesh
- Обзор и сравнение решений
- Istio

# Цели вебинара

## К концу занятия вы:

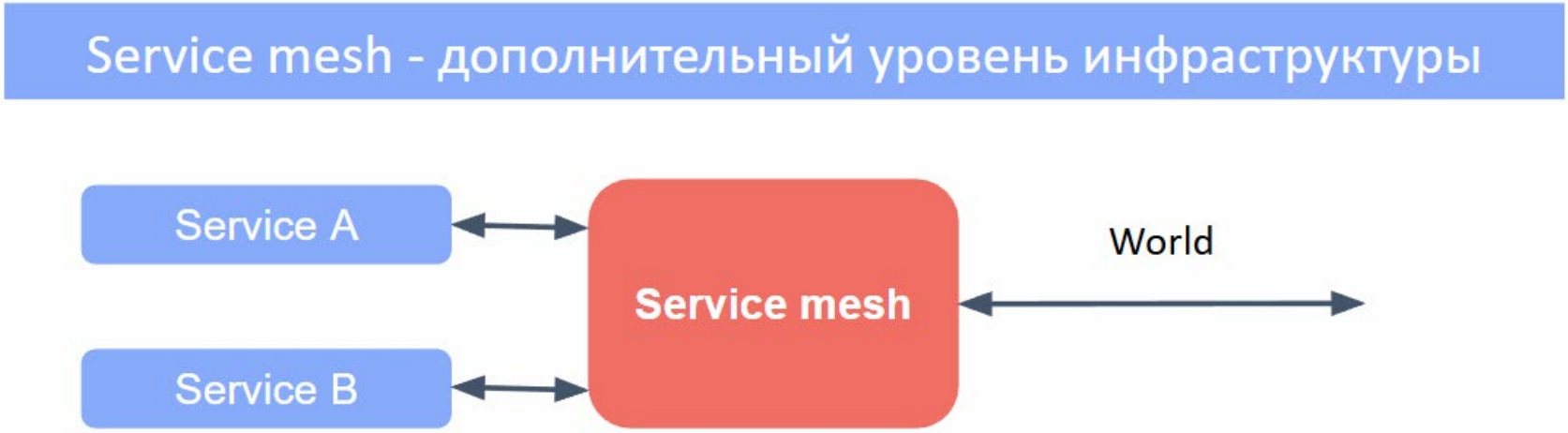
1. Поймете, что такое Service Mesh
2. Узнаете, какие инструменты есть на рынке и чем отличаются
3. Поймете архитектуру Istio
4. Узнаете, какие возможности предоставляет Istio

# Service Mesh

# Service Mesh. Что это?



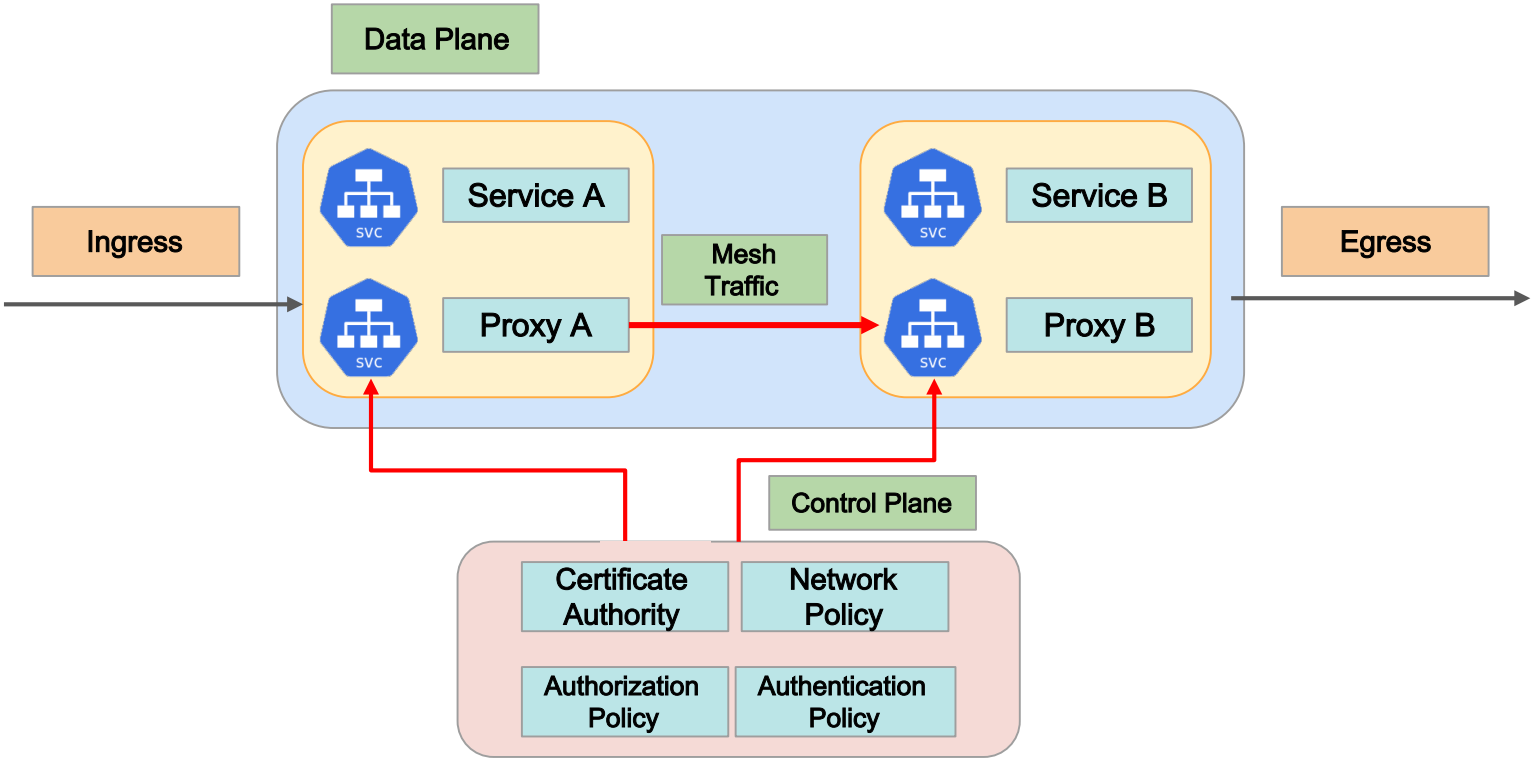
# Service Mesh. Что это?



# Service Mesh. Основной функционал

- Обнаружение сервисов (service discovery)
- Балансировка нагрузки
- Аутентификация и авторизация
- Стратегии деплоя (канареечные релизы)
- Метрики и логирование
- Механизмы observability и tracing
- Retry и circuit breaker
- Chaos engineering
- …

# Service Mesh. Архитектура



# Service Mesh. Control Plane

Управляющий компонент. Единая точка конфигурирования

- Назначение политик
- Распространение политик
- Интеграция с внешней инфраструктурой

# Service Mesh. Control Plane

Архитектурно

- Монолит
- Микросервисный

API

- SMI (Service Mesh Interface)
- GatewayAPI
- Проприетарный

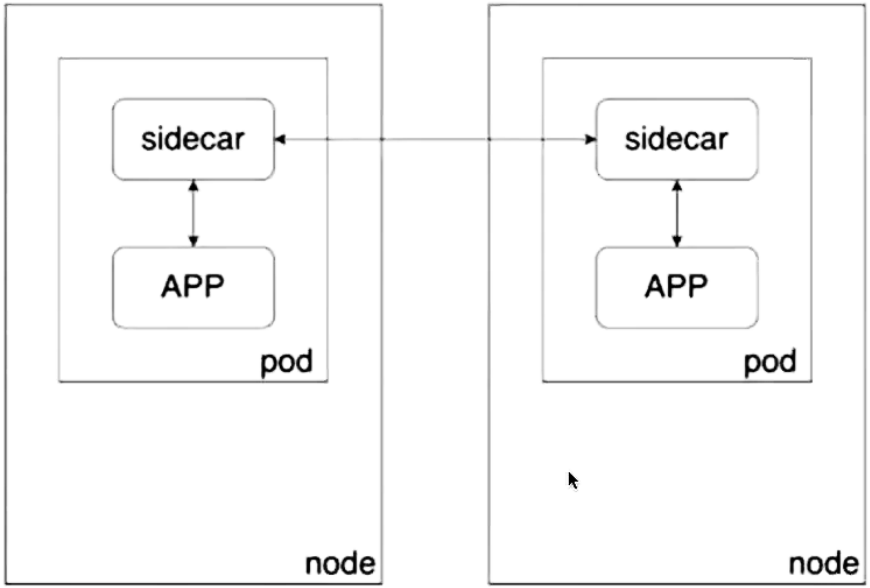
# Service Mesh. Data Plane

- Применение правил настроенных в Control Plane
- Маршрутизация и балансировка трафика
- Аутентификация и авторизация
- Спаны трассировки
- Сбор метрик и логов

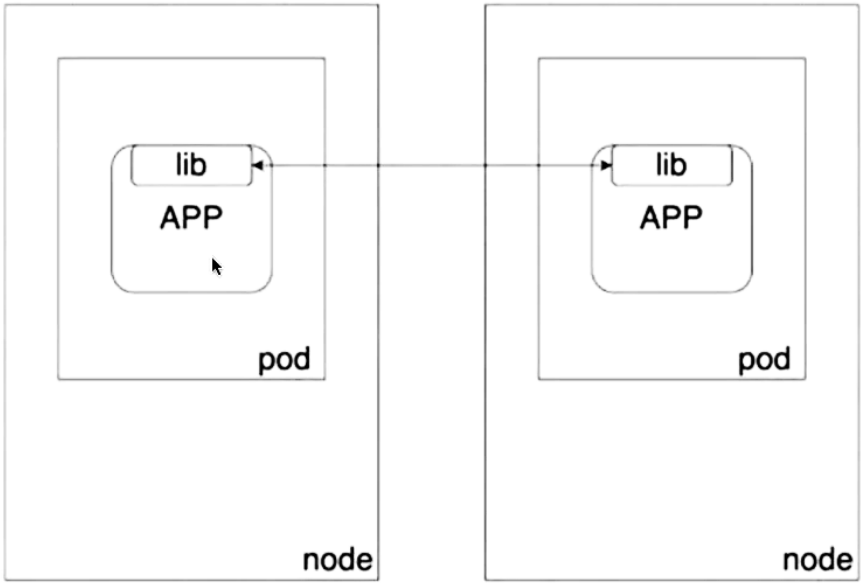
# Service Mesh. Data Plane

- Sidecar proxy
- Proxyless
- Node daemon

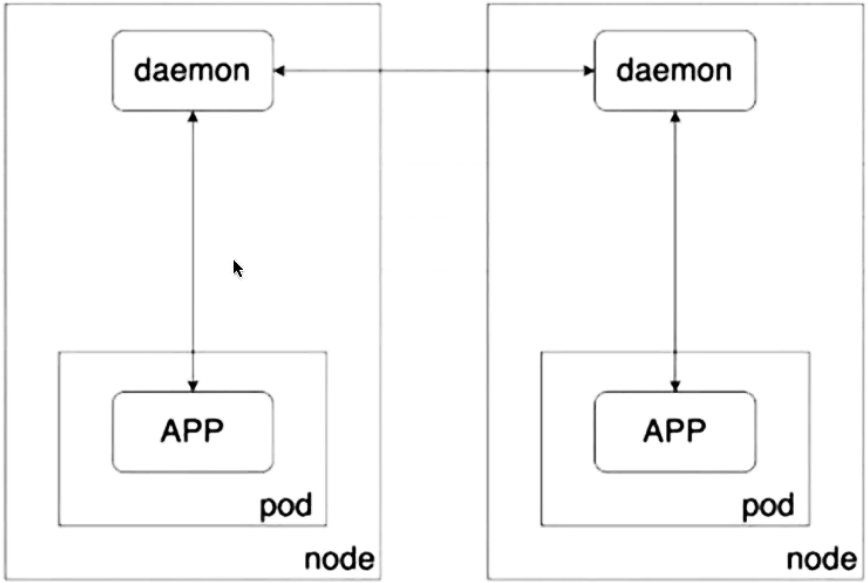
# Service Mesh. Sidecar proxy



# Service Mesh. Proxyless



# Service Mesh. Node daemon



# Service Mesh. Trafic capture

- На уровне пода
- На уровне НОДЫ (CNI)
- eBPF

# Service Mesh. Плюсы

- Внедрение функционала на стороне сервиса и независимо от разработчика
- Гетерогенный стек
- Единая точка конфигурирования

# Service Mesh. Минусы

- Дополнительные накладные расходы на вычислительные ресурсы
- Увеличение latency
- Дополнительная точка отказа
- Сложность развертывания и поддержки

# Вопросы ?

\+ если есть вопросы

– если вопросов нет

# Решения

# Istio

- Монолитный Control Plane
- Data Plane – sidecar, proxyless
- Control Plane API – все виды
- Data Plane API – xDS (Envoy)



# Consul

- Микросервисный Control Plane
- Data Plane – sidecar, proxyless
- Control Plane API – проприетарный, SMI
- Data Plane API – xDS (Envoy)



# Linkerd

- Микросервисный Control Plane
- Data Plane – sidecar
- Control Plane API – все виды
- Data Plane API – проприетарный (Linkerd proxy)



LINKERD

# Cilium Service Mesh

- Микросервисный Control Plane
- Data Plane – Node Daemon
- Control Plane API – проприетарный
- Data Plane API – xDS



# Kuma

- Монолитный Control Plane
- Data Plane – sidecar
- Control Plane API – проприетарный, GW
- Data Plane API – xDS (Envoy)



# Nginx Service Mesh

- Микросервисный Control Plane
- Data Plane – sidecar, proxyless
- Control Plane API – SMI
- Data Plane API – проприетарный (nginx proxy)



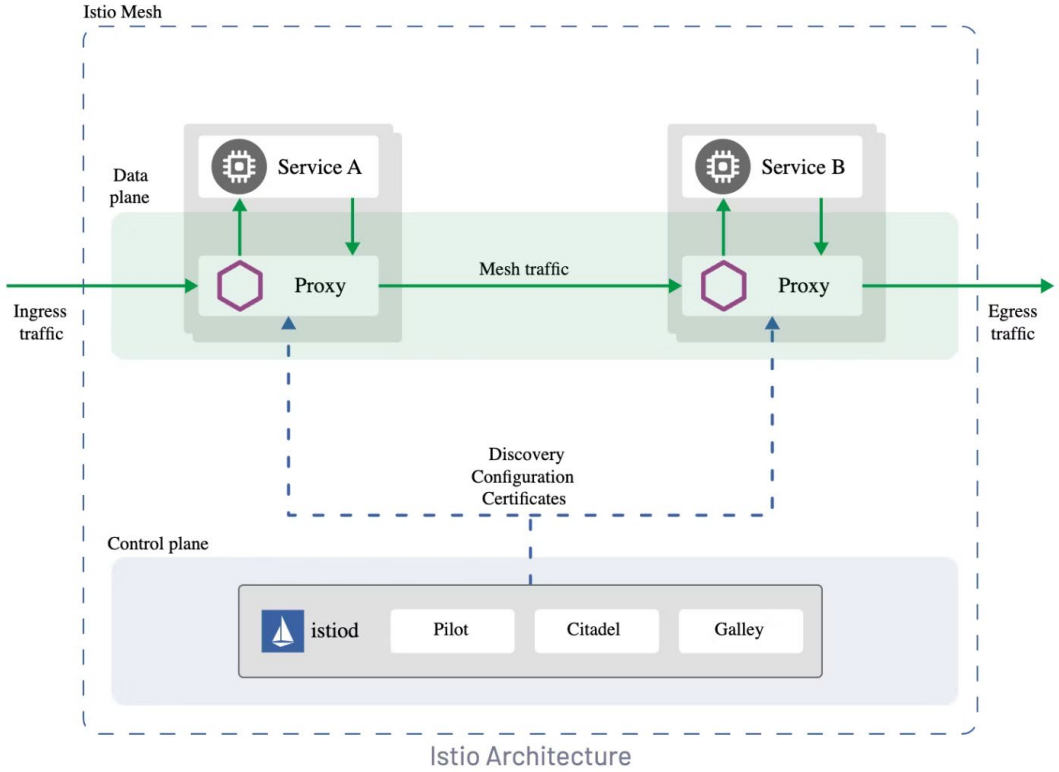
# Вопросы ?

\+ если есть вопросы

– если вопросов нет

# Istio

# Архитектура Istio



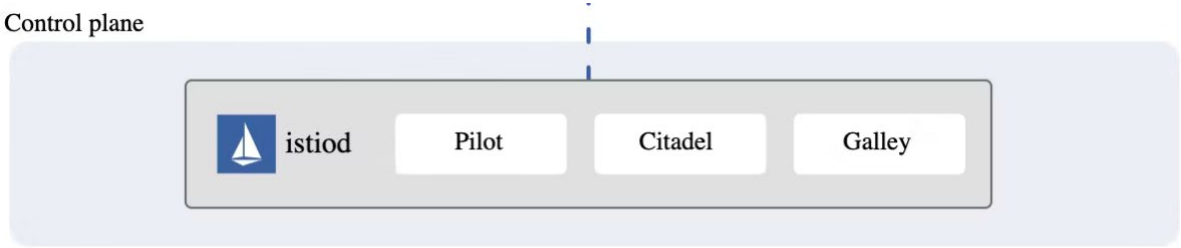
Istio Architecture

# Control Plane. Istiod.

Control plane представляется одним компонентом – Istiod.

Помимо Pilot, Mixer, Citadel включает в себя Galley (агрегация и распределение конфигурации) и Sidecar Injector (config-map)

- Sidecar (т.е. data plane) конфигурируется с помощью Kubernetes CRD (Custom Resource Definitions)
- количество CRD может разниться в зависимости от конфигурации (внутри- или мульти-кластерной)



# Envoy

**Envoy** – это отдельный и самостоятельный opensource -продукт. Является высокопроизводительным прокси-сервером , занимающим небольшой объем оперативной памяти.

Из **основных свойств Envoy** следует выделять:

- Поддержка http/2 и grpc
- Расширенная балансировка нагрузки (повторные попытки, разрыв цепи, глобальное ограничение скорости, затенение запросов, локальную балансировку нагрузки зоны и тд)
- API для динамического управления конфигурацией
- Глубокая наблюдаемость трафика на L7-уровнях
- Поддержка распределенной трассировки

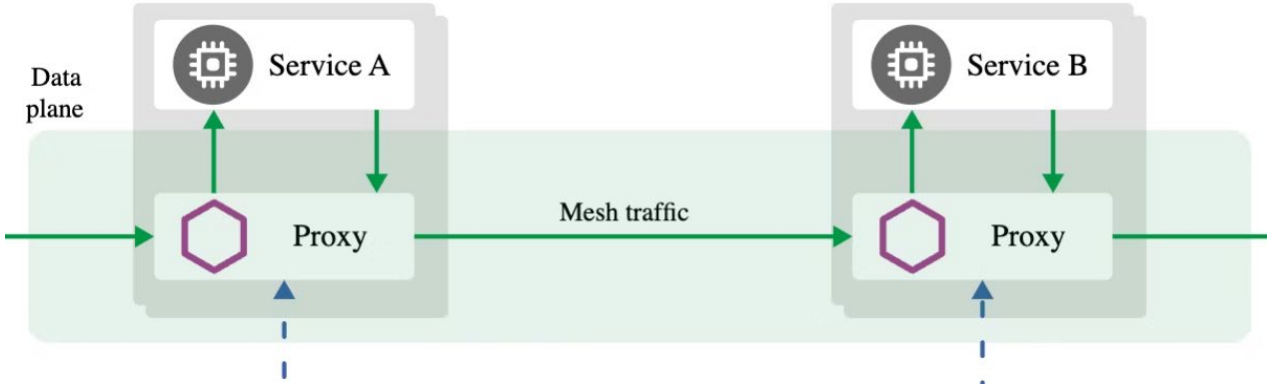


# Data Plane.

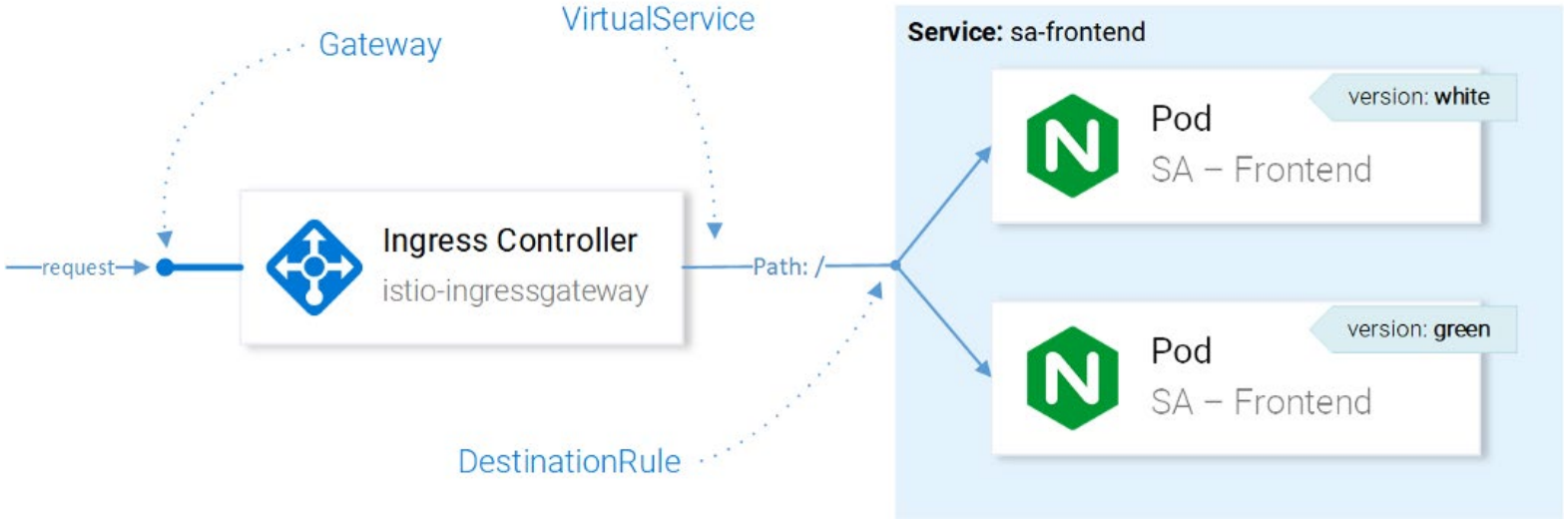
Вставка sidecar'ов может быть произведена автоматически или вручную

- kubectl label namespace default istioinjection=enabled

- istioctl kube-inject -f samples/sleep/sleep.yaml | kubectl apply -f -



# Управление трафиком



# Ingress Gateway

- Разрешает трафик в кластере (по умолчанию Istio блокирует весь входящий трафик, пока не onределён Gateway)
- Istio-Ingressgateway это сервис с типом LoadBalancer
- Внутрикластерная конфигурация – Ingress-gateway
- Мультикластерная конфигурация – east-west-traffic-gateway

# Gateway

- **Gateway** – это CRD (Custom Resource Definition) в Kubernetes, определяемый после установки Istio в кластере и активирующий возможность указывать порты, протоколы и хосты, для которых мы хотим разрешить входящий трафик

```
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: http-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
- "*"
```

Теперь шлюз разрешает доступ к порту 80, но не имеет представления о том, куда маршрутизировать запросы. Для этого понадобятся Virtual Services

# Virtual Service

- Virtual Service указывает Ingress Gateway, как маршрутизировать запросы, которые разрешены внутри кластера

```
kind: VirtualService
metadata:
  name: sa-external-services
spec:
  hosts:
  - "*"
  gateways:
  - http-gateway                    # 1
  http:
  - match:
    - uri:
        exact: /
    - uri:
        exact: /callback
    - uri:
        prefix: /static
    - uri:
        regex: '^.*\.(ico|png|jpg)$'
    route:
    - destination:
        host: sa-frontend           # 2
        port:
number: 80
```

#1: Virtual Service относится к запросам, приходящим через http-gateway, то есть Virtual Service связан с Gateway #2: связан с сервисом, куда отправляются запросы, через раздел destination

# Destination Rule

- **Destination Rule (CRD)**– определяет политики, применяемые к трафику, предназначенному экземплярам сервиса, после того, как его направил сервису Virtual Service

```
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: sa-frontend
spec:
  host: sa-frontend
  trafficPolicy:
    loadBalancer:
      consistentHash:
        httpHeaderName: version    # 1
```

#1: хэш будет генерироваться на основе содержимого HTTP-заголовка version

# Наблюдаемость

- Prometheus (мониторинг, алертинг)
- Kiali (дашборд)
- Grafana (визуализация метрик)
- Jaeger (трассировка)

# Вопросы ?

\+ если есть вопросы

– если вопросов нет

# LIVE

# Кейс

# Вводные

Компания «TravelGo» выросла до 150 микросервисов. Настройка маршрутизации, health-checks и сертификатов стала слишком сложной: разные команды по-разному реализуют проверки, retry-политику и TLS.

DevOps-группа предлагает внедрить **Service Mesh** (например, Istio), чтобы централизовать управление трафиком и наблюдаемостью. Но разработчики переживают, что это усложнит деплой и снизит производительность.

1. Какие задачи решает Service Mesh по сравнению с классическим Service Discovery?
2. Какие плюсы и минусы внедрения Service Mesh?
3. Когда стоит внедрять Service Mesh, а когда он избыточен?
4. Как Service Mesh помогает в health-check и балансировке?
5. Какие организационные изменения могут потребоваться при его внедрении?

# Возможные ответы:

1. *Какие задачи решает Service Mesh по сравнению с классическим Service Discovery?*

   Service Mesh централизует межсервисное взаимодействие —маршрутизацию, балансировку, retries, circuit breakers, TLS, наблюдаемость и политики безопасности.

1. *Какие плюсы и минусы внедрения Service Mesh?*

   Плюсы: единые политики, прозрачная безопасность (mTLS), централизованное управление. Минусы: накладные расходы, кривая обучения, сложность поддержки.

1. *Когда стоит внедрять Service Mesh, а когда он избыточен?*

   Mesh целесообразен при большом количестве микросервисов и необходимости единых политик и наблюдаемости. Для малых систем —избыточен.

1. *Как Service Mesh помогает в health-check и балансировке?*

   Mesh использует sidecar-proxy (Envoy), который реализует health-checks, собирает метрики, балансирует трафик и автоматически исключает недоступные инстансы.

1. *Какие организационные изменения могут потребоваться при его внедрении?*

   Понадобятся новые роли (platform/mesh operator), обновление CI/CD и инфраструктурных пайплайнов, обучение команд.

# Возможное решение:

- Внедрять Service Mesh (например, Istio) при масштабах >50 сервисов, требующих централизованного контроля.
- Настроить стандартные health -checks, retries, circuit breakers и наблюдаемость на уровне mesh.
- Обеспечить автоматическую регистрацию сервисов, TLS между подами и контроль политик.
- Обучить команды и поэтапно включать mesh для отдельных namespaces.

# Вопросы ?

\+ если есть вопросы

– если вопросов нет

# Подведем итоги занятия

# Цели вебинара

Отправьте в чат номера достигнутых целей

1. Поняли, что такое Service Mesh
2. Узнали, какие инструменты есть на рынке и чем отличаются
3. Поняли архитектуру Istio
4. Узнали, какие возможности предоставляет Istio

# Рефлексия



С какими впечатлениями уходите с вебинара?



Как будете применять на практике то, что узнали на вебинаре?

# Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу ✏️❤️

# Приглашаем на следующий вебинар: “Kubernetes Q&A”

Ссылка на вебинар будет в ЛК за 15 минут

Материалы к занятию в ЛК — можно изучать

Обязательный материал обозначен красной лентой